

T3: Разработка Self-Evolving OSINT-Агента (LangGraph + ACE)



Модуль: LangGraph Advanced | **Тайминг:** 1.5 часа

Ключевые технологии: LangGraph · RAG · Exa AI · Jina Reader · Agentic Context Engineering (ACE)



Легенда и задача

Вы разрабатываете **ИИ-профайлера**. Задача агента: собрать глубокое и точное досье на человека по открытым данным (OSINT).

Поскольку в интернете много однофамильцев, скрытых данных и анти-скрапинговых защит, базовый поиск часто ошибается. Ваш агент должен использовать передовой подход **ACE (Agentic Context Engineering)**: он не только ищет данные и обновляет векторную базу (RAG), но и рефлексивирует над своими ошибками поиска, обновляя собственный внутренний **«Плейбук»** - набор правил для будущих итераций.



[Статья: ACE Framework \(arxiv\).](#)

04618v3 [cs.LG] 29 Mar 2026

AGENTIC CONTEXT ENGINEERING: EVOLVING CONTEXTS FOR SELF-IMPROVING LANGUAGE MODELS

Qizheng Zhang^{1*}, Changran Hu^{2*}, Shubhangi Upasani², Boyuan Ma², Fenglu Hong², Vamsidhar Kamanuru², Jay Rainton², Chen Wu², Mengmeng Ji², Hanchen Li³, Urmish Thakker², James Zou¹, Kunle Olukotun¹

¹Stanford University ²SambaNova Systems, Inc. ³UC Berkeley
{qizhengz, kunle}@stanford.edu changran_hu@berkeley.edu
🔗 ace-agent/ace 🌐 ace-agent.github.io *Equal contribution.

ABSTRACT

Large language model (LLM) applications such as agents and domain-specific reasoning increasingly rely on *context adaptation*: modifying inputs with instructions, strategies, or evidence, rather than weight updates. Prior approaches improve usability but often suffer from brevity bias, which drops domain insights for concise summaries, and from context collapse, where iterative rewriting erodes details over time. We introduce ACE (Agentic Context Engineering), a framework that treats contexts as evolving playbooks that accumulate, refine, and organize strategies through a modular process of generation, reflection, and curation. ACE prevents collapse with structured, incremental updates that preserve detailed knowledge and scale with long-context models. Across agent and domain-specific benchmarks, ACE optimizes contexts both offline (*e.g.*, system prompts) and online (*e.g.*, agent memory), consistently outperforming strong baselines: +10.6% on agents and +8.6% on finance, while significantly reducing adaptation latency and rollout cost. Notably, ACE could adapt effectively without labeled supervision and instead by leveraging natural execution feedback. On the AppWorld leaderboard, ACE matches the top-ranked production-level agent on the overall average and surpasses it on the harder test-challenge split, despite using a smaller open-source model. These results show that comprehensive, evolving contexts enable scalable, efficient, and self-improving LLM systems with low overhead.

🔧 Архитектура инструментов (Tools)

Вам необходимо обернуть в `@tool` (или замочать) следующие три функции:

Инструмент	Описание	Ссылка
 Exa_Search	Семантический поиск. Ищет упоминания, статьи и соцсети по ФИО, является и поисковиком и парсером	exa.ai
 Jina_Reader_Scraper	Принимает URL от Exa, обходит защиту сайтов и возвращает чистый Markdown страницы.	jina.ai/reader
 Vector_DB_Manager (RAG)	<code>query_profile(name)</code> • проверяет наличие досье. <code>upsert_profile(name, facts)</code> •	/

Инструмент	Описание	Ссылка
	создаёт или мержит факты с разрешением конфликтов.	

Логика графа (ACE Framework)

Вместо классического цикла «сгенерировал → проверил», мы реализуем **триаду ролей**: Generator, Reflector, Curator. Агент учится на лету - не меняя веса модели, а адаптируя свой контекст.

1. Структура State

Ваш `StateGraph` должен опираться на следующий `TypedDict`:

```
from typing import TypedDict, Annotated, List
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    target_person: str          # Кого ищем (ФИО + базовый контекст)
    playbook: List[str]        # Ключевая фишка ACE: список правил поиска
    insights: str               # Выводы после попытки поиска (от Reflector)
    messages: Annotated[list, add_messages] # История вызовов в тулов
    iterations: int            # Счётчик циклов (защита от бесконечного лупа)
```

2. Узлы (Nodes)

▼ Generator - Исполнитель

LLM получает `target_person` и текущий `playbook`. Принимает решение, какие запросы отправить в Еха и какие ссылки отдать Jina. Формирует структурированную сводку извлечённых фактов.

▼ 📁 RAG_Upsert

Сохраняет или дополняет собранные факты в векторную БД.

▼ 🧐 Reflector - Критик / Аналитик

Анализирует работу Генератора и выдаёт конкретные `insights` (уроки).

Пример: «Мы искали "Иван Иванов" и Jina принёс статью про строителя, а нам нужен был AI-инженер. Мы забыли добавить контекст профессии в Eха».

▼ 📝 Curator - Куратор контекста

Берёт `insights` от Рефлектора и **инкрементально** обновляет `playbook`.

Важно: он не переписывает системный промпт целиком, а добавляет новое точечное правило (например: «Для этого человека всегда добавляй в поисковой запрос "AI, Machine Learning"») - либо удаляет нерабочие стратегии.

3. Поток выполнения (Workflow)



В чём главная сила LangGraph?

Обычный пайплайн ищет один раз и останавливается. LangGraph позволяет агенту **накапливать контекст между итерациями** и использовать найденное как трамплин для следующего поиска.

Пример: ищем Армана, который работает в Infactorial.

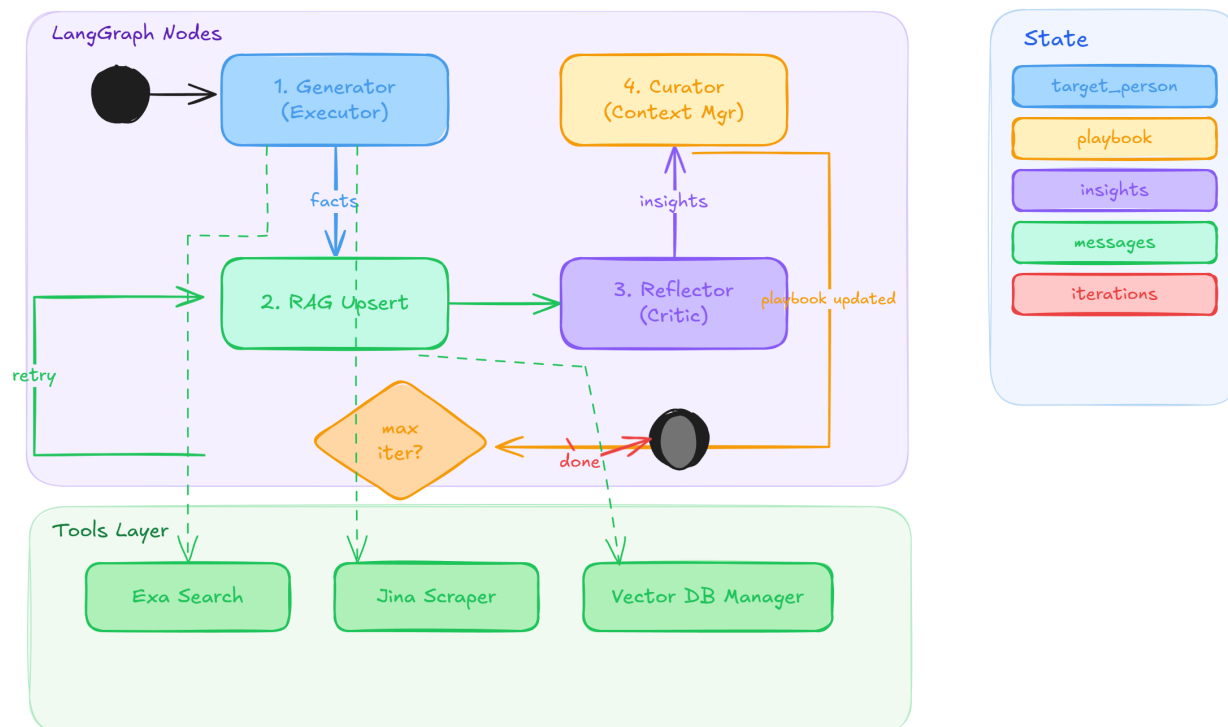
- **Итерация 1:** Generator делает запрос `"Арман Сулейменов nfactorial"`. Jina возвращает страницу с упоминанием, что он Senior Engineer в Infactorial.
- **RAG_Upsert** сохраняет факт: `"связан с компанией nfactorial"`.
- **Reflector** замечает: досье поверхностное, мало личных данных.
- **Curator** обновляет playbook: `"Искать также по окружению: коллеги из nfactorial, LinkedIn компании, упоминания на GitHub"`.
- **Итерация 2:** Generator уже ищет не только самого человека, но и его окружение - коллег, проекты, публичные репозитории компании.

Вот это и есть ключевое отличие от простого веб-поиска: агент **расширяет граф знаний**, двигаясь от человека к его связям.

1. Запуск с пустым (или базовым) `playbook`.
2. **Generator** → **RAG_Upsert**
3. **Reflector** проверяет качество поиска.
4. Если есть ошибки или недостаток данных → **Curator** обновляет `playbook` → возврат к **Generator**.
5. Если данные исчерпывающие или достигнут лимит `iterations` → **END**.

ACE Profiler — LangGraph

Agentic Context Engineering



✓ Чек-лист выполнения

- ☐ Определить класс `AgentState`.
- ☐ Инициализировать 3 инструмента (`Exa_Search`, `Jina_Reader_Scraper`, `Vector_DB_Manager`).
- ☐ Написать системные промпты для узлов `Generator`, `Reflector`, `Curator`.
- ☐ Реализовать логику узлов (Python-функции).
- ☐ Собрать `StateGraph`, настроить условные рёбра (Conditional Edges) от `Reflector` к `Curator` и `Generator`.
- ☐ **Финальный тест:** Запустить агента на поиск человека с распространённым именем и узкой специальностью. Вывести в консоль `state['playbook']` на каждой итерации, чтобы увидеть процесс эволюции правил.